

(P NEEHARIKA-2520090233)

SKILL WEEK 7 - SESSION 13

TASK 1: To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int file;
    int saved_stdin;
    char line[100];

    printf("Input Redirection Demonstration\n");

    FILE *fp = fopen("input.txt", "w");

    if (fp == NULL) {
        perror("fopen");
        return 1;
    }

    fprintf(fp, "First line from input file\n");
    fprintf(fp, "Second line from input file\n");
    fclose(fp);

    file = open("input.txt", O_RDONLY);

    if (file == -1) {
        perror("input.txt");
        return 1;
    }

    saved_stdin = dup(STDIN_FILENO);

    dup2(file, STDIN_FILENO);
    close(file);

    printf("Reading redirected input:\n");

    while (fgets(line, sizeof(line), stdin) != NULL) {
        printf("%s", line);
    }

    dup2(saved_stdin, STDIN_FILENO);
    close(saved_stdin);

    printf("Input stream restored successfully.\n");

    return 0;
}
```

```
root@HP: ~/skill_week7
root@HP:~/skill_week7# gcc task1_input_redirect.c -o task1_input_redirect
root@HP:~/skill_week7# ./task1_input_redirect
Input Redirection Demonstration
Reading redirected input:
First line from input file
Second line from input file
Input stream restored successfully.
root@HP:~/skill_week7#
```

Task 2: To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test

```
root@HP: ~/skill_week7 task2_output_redirect.c *
GNU nano 7.2
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int file;
    int saved_stdout;

    printf("Output Redirection Demonstration\n");

    file = open("output.txt",
                O_WRONLY | O_CREAT | O_TRUNC,
                0644);

    if (file == -1) {
        perror("output.txt");
        return 1;
    }

    saved_stdout = dup(STDOUT_FILENO);

    dup2(file, STDOUT_FILENO);
    close(file);

    printf("This message is redirected to output.txt\n");
    printf("Output redirection completed successfully.\n");

    fflush(stdout);

    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);

    printf("stdout restored successfully.\n");

    return 0;
}
```

Redirection

```
root@HP:~/skill_week7# nano task2_output_redirect.c
root@HP:~/skill_week7# gcc task2_output_redirect.c -o task2_output_redirect
root@HP:~/skill_week7# ./task2_output_redirect
Output Redirection Demonstration
stdout restored successfully.
root@HP:~/skill_week7# ./task2_output_redirect
Output Redirection Demonstration
stdout restored successfully.
root@HP:~/skill_week7# cat output.txt
This message is redirected to output.txt
Output redirection completed successfully.
root@HP:~/skill_week7#
```

SESSION 14

Task 1

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int file;
    int saved_stdout;

    file = open("append.txt",
               O_WRONLY | O_CREAT | O_APPEND,
               0644);

    if (file == -1) {
        perror("append.txt");
        return 1;
    }

    saved_stdout = dup(STDOUT_FILENO);

    dup2(file, STDOUT_FILENO);
    close(file);

    printf("New data appended to the file.\n");

    fflush(stdout);

    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);

    printf("Append operation completed.\n");

    return 0;
}
```

Scenarios

```
root@HP:~# cd ~/skill_week7
root@HP:~/skill_week7# gcc task3_append.c -o task3_append
root@HP:~/skill_week7# ./task3_append
Append operation completed.
root@HP:~/skill_week7# ./task3_append
Append operation completed.
root@HP:~/skill_week7# cat append.txt
New data appended to the file.
New data appended to the file.
New data appended to the file.
New data appended to the file.
New data appended to the file.
root@HP:~/skill_week7#
```

Task 2

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Scenarios

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int file;
    int saved_stderr;

    file = open("error.txt",
               O_WRONLY | O_CREAT | O_TRUNC,
               0644);

    if (file == -1) {
        perror("error.txt");
        return 1;
    }

    saved_stderr = dup(STDERR_FILENO);

    dup2(file, STDERR_FILENO);
    close(file);

    fprintf(stderr, "This is an error message.\n");
    fprintf(stderr, "Error output was redirected successfully.\n");

    fflush(stderr);

    dup2(saved_stderr, STDERR_FILENO);
    close(saved_stderr);

    printf("stderr restored successfully.\n");

    return 0;
}

root@HP:~/skill_week7# gcc task4_stderr_redirect.c -o task4_stderr_redirect
root@HP:~/skill_week7# ./task4_stderr_redirect
stderr restored successfully.
root@HP:~/skill_week7# cat error.txt
This is an error message.
Error output was redirected successfully.
root@HP:~/skill_week7#
```

Observation — Session 13

Input and output redirection were implemented successfully using file descriptors, and the redirected data was verified through files.

Observation — Session 14

Append and stderr redirection were implemented successfully, with existing data preserved and error output captured and verified.